Lab 3 Report: Trajectory Tracking with a Geometric Controller

Xuanye Xu

Student ID 24020036071 / H00490299 Group 7

Robotics Integration I — VNAV Labs

October 19, 2026

A geometric tracking controller on SE(3) was implemented for the simulated quadrotor of the MIT VNAV Lab 3 and tuned to follow a circle of radius 5 m at 2 m altitude. With the final gains $(k_x, k_v, k_R, k_\Omega) = (8, 5, 10, 2)$ the position error is 4.3 cm RMS and the pitch jitter 0.26°, and the drone's yaw trails the rotating `av-desired` frame by 5.86°, close to the 5.74° predicted from the control law. Gains chosen on tracking error alone reached 1.8 cm but made the rotors chatter and reverse, and were rejected.

Grading Map

Part	Grading item	Points	Section	Evidence
Individual	Deliverable 1: transformations	25	Section 2	One-line answers; output of a check program
Individual	Deliverable 2: modelling and control	25	Section 3	F matrices and their ranks; drag compensation
Team	ROS communication	10	Section 4	Node graph; topic rates; frame of the odometry twist
Team	Controller (code body)	20	Section 5	Control law and code; circle tracking; RViz video
Team	`av-desired` rotation	10	Section 6	RViz sequence; yaw tracking and a model of its lag
Team	Gain tuning	10	Section 7	36 logged runs; error and smoothness table; video

1 *Lab Introduction and Team Information*

The lab implements the geometric controller of Lee et al.,[1] which turns a desired trajectory (position, velocity, acceleration and yaw) into the four rotor speeds of a quadrotor in the TESSE Unity simulator, tunes it on a circular trajectory, and adds two individual exercises on rotation conventions and quadrotor modelling.[2] The code is the `VNAV2023` release of the lab template.[3] Group 7 has one member; the code, the experiments and the draft of this report were done with the help of an AI assistant (Claude, by Anthropic),[4] and the author reviewed all results reported here. Sections 2 and 3 are the individual deliverables and Sections 4 to 7 the team experiment.

[1] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010

[2] MIT SPARK Lab. VNAV 2023 Lab 3: Trajectory Tracking for UAVs. https://vnav.mit.edu/labs_2023/lab3/exercises.html, 2023a

[3] MIT SPARK Lab. VNAV-labs, release VNAV2023. https://github.com/MIT-SPARK/VNAV-labs/releases/tag/VNAV2023, 2023b

The course page links the 2021 release, which for this lab differs only in the move of the trajectory publisher to `tf2` and one Python requirement; the downloadable simulator matches 2023. Environment: WSL 2 Ubuntu 20.04.6, ROS Noetic, simulator rendering on an RTX 4070 SUPER through D3D12. Commands that reproduce every item: `lab3/HUMANS.md`.

[4] Anthropic. Claude opus 5.5 (ai assistant). https://www.anthropic.com/claude, 2026

2 *Deliverable 1: Transformations in Practice* [25 pts]

The deliverable asks for one-line conversions between a `geometry_msgs::Quaternion` and a `tf2::Quaternion`, the scalar part of a quaternion, its yaw, and a 3×3 Eigen rotation matrix built from a message. The answers (from the headers `tf2_geometry_msgs`, `tf2/utils.h` and `tf2_eigen`):

```
tf2::fromMsg(quat_msg, quat_tf);                          // A.1
geometry_msgs::Quaternion quat_msg = tf2::toMsg(quat_tf); // A.2
quat_tf.getW();                                           // A.3
tf2::getYaw(quat_t);                                      // B.1
Eigen::Quaterniond q;  tf2::fromMsg(quat_msg, q);         // B.2
Eigen::Matrix3d R = q.toRotationMatrix();
```

A check program runs every line on the rotation with roll, pitch and yaw $(0.1, -0.2, 0.7)$ rad: the quaternion survives the round trip, `getYaw` returns 0.700000, and the Eigen matrix matches `tf2::Matrix3x3` exactly.

> EVIDENCE `lab3/scripts/d1_tf2_check.cpp` and its output `lab3/results/logs/d1_tf2_check.txt`.

3 *Deliverable 2: Modelling and Control of UAVs* [25 pts]

(A) Structure of quadrotors. Rotor i at (x_i, y_i), spinning in direction $s_i = \pm 1$ ($+1$ counter-clockwise seen from above), adds $c_f u_i$ to the thrust, $c_f y_i u_i$ and $-c_f x_i u_i$ to the roll and pitch torques and $-s_i c_d u_i$ to the yaw torque, with $u_i = \omega_i |\omega_i|$; so $(f, \tau_x, \tau_y, \tau_z)^\top = F\, u$. In the figure of the exercise the rotors sit at $(x_i, y_i) = a\,(-1, 1)$, $a\,(-1, -1)$, $a\,(1, -1)$, $a\,(1, 1)$ with $a = d/\sqrt{2}$, and quadrotor (a) has $s = (1, -1, 1, -1)$:

$$F_a = \operatorname{diag}(c_f,\, a c_f,\, a c_f,\, c_d) \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}.$$

The rows of the sign matrix are mutually orthogonal, so F_a is invertible: it has rank 4 and every wrench $(f, \tau_x, \tau_y, \tau_z)$ can be produced. In quadrotor (b) rotors 3 and 4 are reversed, $s = (1, -1, -1, 1)$, and the last row becomes $c_d\,(-1, 1, 1, -1)$, a multiple of the second row. Hence rank $F_b = 3$ and, whatever the rotor speeds,

$$\tau_z = -\frac{c_d}{a\, c_f}\, \tau_x.$$

Tracking x, y, z needs the thrust and the two tilting torques, and tracking the yaw ψ needs a yaw torque that can be chosen on its own. In (b) the yaw torque is tied to the roll torque: the drone cannot turn without rolling, and a pure yaw torque is out of reach, so a trajectory in $[x, y, z, \psi]$ cannot be tracked.

Numerical check with $c_f = 10^{-3}$, $c_d = 10^{-5}$, $d = 0.3\,\text{m}$: the singular values of F_b end in 3.6×10^{-21}, the yaw row is -0.0471 times the roll row, and the best least-squares approximation of a pure yaw torque leaves a relative residual of 0.999.

(B) Control of quadrotors. With the body drag $F^b = D\,(v^b)^2$, where $D = \mathrm{diag}(0.1, 0.1, 0.2)$, $v^b = R^\top v$ and $(v^b)^2 = -(v_x^b|v_x^b|,\, v_y^b|v_y^b|,\, v_z^b|v_z^b|)$ opposes the motion, the translational dynamics (z up) become $m\dot{v} = fRe_3 - mg\,e_3 + R\,F^b$. The controller forms the desired thrust vector $A = -k_x e_x - k_v e_v + mg\,e_3 + m\ddot{x}_d$ and uses it twice, for the desired body axis $b_{3d} = A/\|A\|$ and for the thrust $f = A \cdot Re_3$. Subtracting the predicted drag,

$$A' = -k_x e_x - k_v e_v + mg\,e_3 + m\ddot{x}_d - R\,F^b\left(R^\top v\right),$$

and using A' in both places cancels the drag: once $Re_3 = b_{3d}$, $m\ddot{e}_x = -k_x e_x - k_v e_v$ as without drag. The drag can be evaluated at the measured velocity (exact, but noisier) or as a feed-forward at the desired one, $R_d F^b(R_d^\top v_d)$. On the lab's circle the drone always flies along its own x axis at 2.5 m/s, since its yaw turns with the velocity, so the drag would be a constant 0.63 N leaving, uncompensated, an error of about $0.63/k_x \approx 8$ cm. This is a derivation only; it was not tested in the simulator, whose drag model is not documented.

> EVIDENCE `lab3/scripts/d2_rank_check.py` and its output `lab3/results/logs/d2_rank_check.txt`.

4 *ROS Communication* [10 pts]

Parts 1 and 2 of `controller_node.cpp` declare and create two subscribers, a publisher and a timer (Figure 1): `/desired_state` (`MultiDOFJointTrajectoryPoint`, from `traj_publisher`), `/current_state` (`nav_msgs/Odometry`, from the bridge) and `/rotor_speed_cmds` (`mav_msgs/Actuators`, to the bridge's control interface). Parts 3 and 4 copy $x_d, v_d, \ddot{x}_d, \psi_d$ and x, v, R, Ω out of the messages. The one step that needs care is the angular velocity: the controller wants Ω in the body frame, but the odometry twist is given in the world frame. The template says so, and recorded data confirm it: against finite differences of the pose, the reported angular velocity fits a world-frame reading in all four recordings (relative error 6–47 %, against 74–117 % for the body frame), so the code uses $\Omega = R^\top \omega_{\text{world}}$. The timer is set to 1 kHz (template default) on the simulated clock; the measured rates are 99.8 Hz for `/desired_state`, 50.0 Hz for `/current_state` and 97.5 Hz for `/rotor_speed_cmds`.

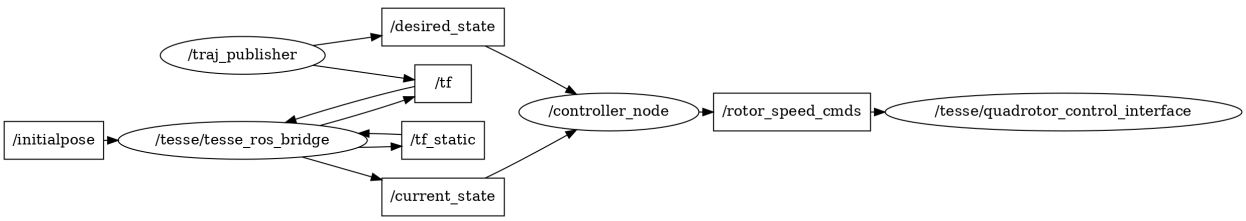


Figure 1: Node graph during circle tracking: the controller reads the desired and current states and drives the simulator through `/rotor_speed_cmds`.

> EVIDENCE `lab3/results/screenshots/rqt_graph.png`; `rosnode info` and `rostopic hz` in `lab3/results/logs/ros_communication.txt`; frame test in `lab3/results/logs/frame_check.txt`.

5 *Controller Implementation* [20 pts]

The control law follows Lee et al.[5] with the lab's conventions. The z axes point up (the paper's point down, which flips the signs of gravity and of b_{3d}); the body x axis lies halfway between two arms, so the rotor map is the F of Section 3 with the simulator's rotor numbering instead of the paper's eq. (1); and $c_{\tau f} = c_d/c_f$. Each cycle computes

$$e_x = x - x_d, \quad e_v = v - v_d, \quad A = -k_x e_x - k_v e_v + mg\,e_3 + m\ddot{x}_d,$$
$$b_{3d} = \frac{A}{\|A\|}, \quad b_{2d} = \frac{b_{3d} \times b_{1c}}{\|b_{3d} \times b_{1c}\|}, \quad b_{1d} = b_{2d} \times b_{3d},$$
$$R_d = [\, b_{1d}\ b_{2d}\ b_{3d}\,], \quad e_R = \tfrac{1}{2}\left(R_d^\top R - R^\top R_d\right)^\vee, \quad e_\Omega = \Omega,$$
$$f = A \cdot R e_3, \quad M = -k_R e_R - k_\Omega e_\Omega + \Omega \times J\Omega,$$
$$u = F^{-1}\begin{bmatrix} f \\ M \end{bmatrix}, \quad \omega_i = \operatorname{sign}(u_i)\sqrt{|u_i|},$$

with $b_{1c} = (\cos\psi_d, \sin\psi_d, 0)$. As the template allows, the terms with the desired angular velocity Ω_d and its derivative are dropped ($e_\Omega = \Omega$); Section 6 shows what this costs. The core of Part 5:

```
const double l = d * cf / sqrt(2.0);   // rotors 1..4 in body quadrants I..IV
F2W << cf, cf, cf, cf,   l, l, -l, -l,   -l, l, l, -l,   cd, -cd, cd, -cd;

const Eigen::Vector3d thrust_des = -kx * ex - kv * ev + m * g * e3 + m * ad;
const Eigen::Vector3d b3d = thrust_des.normalized();
const Eigen::Vector3d b1d_yaw(cos(yawd), sin(yawd), 0.0);
const Eigen::Vector3d b2d = b3d.cross(b1d_yaw).normalized();
const Eigen::Vector3d b1d = b2d.cross(b3d);
Eigen::Matrix3d Rd;  Rd << b1d, b2d, b3d;
er = 0.5 * Vee(Rd.transpose() * R - R.transpose() * Rd);
eomega = use_omega_d ? Eigen::Vector3d(omega - R.transpose() * omegad_world)
                     : omega;            // use_omega_d = false in Lab 3
const double f = thrust_des.dot(R * e3);
const Eigen::Vector3d M = -kr * er - komega * eomega + omega.cross(J * omega);
Eigen::Vector4d wrench;  wrench << f, M;
const Eigen::Vector4d w2 = F2W.inverse() * wrench;  // omega_i * |omega_i|
for (int i = 0; i < 4; ++i) cmd.angular_velocities[i] = signed_sqrt(w2(i));
```

The controller was first tested hovering at $(0, 0, 2\,\text{m})$ with yaw 45°, then on the circle. With the final gains (Figure 2), over 40 s (three laps) the position error is 4.3 cm RMS and at most 4.8 cm. Most of it is a constant altitude offset: the mean altitude is 1.958 m instead of 2.000 m. The offset comes from the control law, which has no integral term. In the 14 hover runs that settled (three others oscillated) the sag times k_x lies between 0.36 N and 0.42 N (for $k_x = 1$ the sag is 36 cm, for $k_x = 20$ 1.8 cm): a constant force of about 4 % of the weight is missing from the model, for instance through a slightly different mass or thrust coefficient, and the position "spring" k_x stretches until it supplies that force.

[5] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010

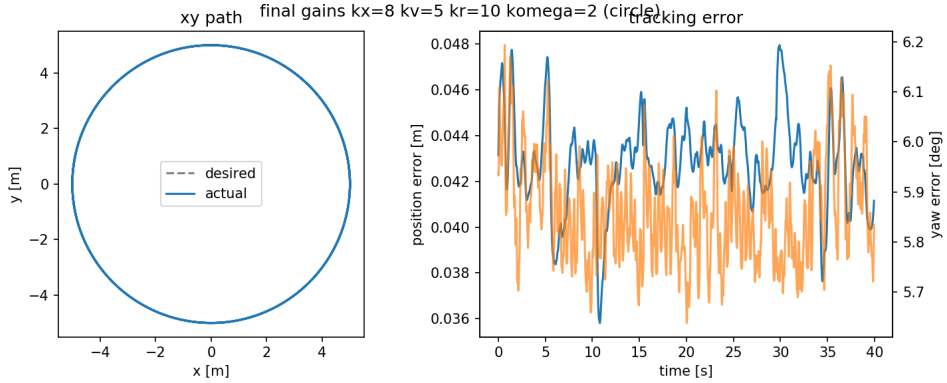


Figure 2: Final gains on the circle. Left: desired and actual paths, which coincide at this scale. Right: position error (blue, left axis) and yaw error (orange, right axis) over 40 s.

> EVIDENCE `lab3/controller_pkg/src/controller_node.cpp`; run summary `lab3/results/logs/final_circle_summary.txt`; one lap in RViz, `lab3/results/video/rviz_circle.mp4`; simulator view `lab3/results/screenshots/sim_circle.png`.

6 *Rotation of* `av-desired` [10 pts]

`traj_publisher` broadcasts the desired pose as the TF frame `av-desired`; its yaw turns at a constant $-0.5\,\mathrm{rad/s}$ while the frame moves round the circle. The grading item can be read in two ways, and both are shown: the frame rotates as it moves (Figure 3), and the drone's own frame `base_link_gt` rotates with it (Figure 4). Over 40 s the desired yaw turns by -1144° and the drone follows with a constant offset of $5.86(10)^\circ$.

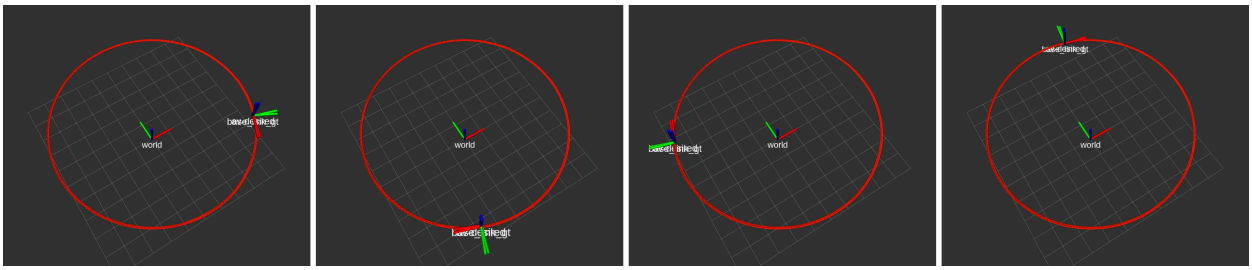


Figure 3: RViz top view at four points of one lap. `av-desired` and `base_link_gt` overlap, so their labels print on top of each other; the red x axis of the pair turns with the position on the circle.

The offset follows from the control law. With $e_\Omega = \Omega$, a drone that turns at the desired rate still meets the damping torque $-k_\Omega \Omega_z$, which in steady state the attitude term must balance; for a pure yaw error $\Delta\psi$ the yaw component of $-k_R e_R$ is $-k_R \sin\Delta\psi$. Hence

$$\sin\Delta\psi = \frac{k_\Omega\,|\dot{\psi}_d|}{k_R}.$$

This predicts 5.74° for the final gains (measured 5.86°) and 3.82° for $(20, 7, 30, 4)$ (measured 3.99°); the small remainder is consistent with some yaw drag in the simulator. Feeding Ω_d forward, $e_\Omega = \Omega - R^\top R_d \Omega_d$, removes the lag; it was added as the option `use_omega_d` for Lab 4.

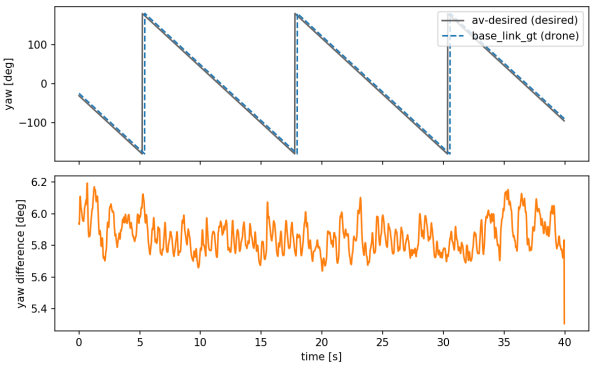


Figure 4: Top: yaw of `av-desired` and of the drone (wrapped to ±180°). Bottom: their difference.

> EVIDENCE `rviz_q1..q4.png` in `lab3/results/screenshots`; `lab3/results/logs/av_desired_yaw.txt` and `lab3/results/logs/av_desired_yaw_oldgains.txt`.

7 *Gain Tuning* [10 pts]

A script respawned the drone, flew it with given gains, recorded a rosbag and scored the run; 36 runs were logged, 17 hovering and 19 on the circle. Besides the tracking error the score includes smoothness: the jitter (RMS of roll and pitch minus their 0.5 s moving average), the RMS step between consecutive rotor commands, and the mean rotor speed. Hovering needs about $\sqrt{mg/4c_f} \approx 50\,\mathrm{rad/s}$ per rotor, so a much lower mean means that the commands keep changing sign.

1. *Hover, inner loop.* With $k_x = k_v = 1$, raising k_R from 1 to 20 left the 36 cm sag unchanged, since the two loops do different jobs; $k_\Omega = 5$ made the rotor commands swing between $\pm 970\,\mathrm{rad/s}$, negative half of the time.

2. *Hover, outer loop.* Raising k_x cut the sag as $1/k_x$ (Section 5). A stronger outer loop needs a faster inner one: with $(k_x, k_v) = (10, 5)$ the drone swung at 1 Hz with $(k_R, k_\Omega) = (10, 1)$ and held its position to 3.6 cm with $(20, 3)$.

3. *Circle, round one*, scored on the tracking error alone: the error kept falling as all gains rose, and the best set, $(20, 7, 30, 4)$, reached 1.8 cm RMS in a 40 s run.

4. *Round two*, scored on error and smoothness: $(20, 7, 30, 4)$ jitters 1.39° in pitch and its rotors average 17.8 rad/s, so they keep reversing, which a real motor cannot do. The final gains cut the jitter to 0.26° and keep the rotors near the hover speed, at the cost of 2.5 cm more error (Table 1).

> EVIDENCE All runs in `lab3/results/logs/tuning.txt`; smoothness in `lab3/results/logs/jitter_comparison.txt`;

$(k_x, k_v, k_R, k_\Omega)$	error cm	jitter deg	step rad/s	mean rad/s
$(1,1,1,1)$ template	diverges	–	–	–
$(20,7,30,4)$ round one	1.8	1.39	80.3	17.8
$(8,5,10,2)$ final	4.3	0.26	21.0	46.4

Table 1: Circle runs of 40 s: position error RMS, pitch jitter, rotor command step RMS and mean rotor speed. With the template gains the drone leaves the circle (error 25 m RMS).

round-one and final gains side by side in `lab3/results/video/sim_old_vs_new.mp4`; a batch discarded for lack of respawns, `lab3/results/logs/tuning_invalid_no_respawn.txt`.

8 *Personal Reflection*

What I take from this lab is how a quadrotor moves at all. Its four rotors can only push along the body z axis: changing their speeds changes how hard they push and at which corner, but never produces a force along x or y. To move in the horizontal plane the drone therefore has to tilt first, so that part of the thrust points the way it should go while the vertical part still balances gravity, and only then can it fly forward steadily. This is also why the controller works in two stages, the position part choosing where the thrust should point and the attitude part tilting the body to point it there. Nothing in the experiment contradicted this picture.

For a real drone I would add a limit that the simulator does not impose: the rotor speeds must not get too low. Each rotor has to keep spinning fast enough to carry its share of the weight, about 50 rad/s here, and running a motor at very low speed, let alone reversing it, would hurt its performance and its lifetime. By this measure the round-one gains $(20,7,30,4)$ would be unusable, since their rotors averaged 17.8 rad/s and kept reversing, while with the final gains the rotors stay close to the hover speed.

9 *Discussion and Conclusions*

With gains chosen on error and smoothness the controller tracks the circle with 4.3 cm RMS error and smooth rotor commands. The law has no integral term, which leaves the 4 cm altitude offset; an integral of e_x in A would remove it.

Problems met under WSLg: windows show on the Windows desktop only when WSL is started from the desktop session; the respawn key reaches Unity only through `xdotool key --window`; `ffmpeg -f x11grab` records black frames, so videos were assembled from frames grabbed window by window; and the bridge scripts needed `python3` first lines.

References

Anthropic. Claude opus 5.5 (ai assistant). https://www.anthropic.com/claude, 2026.

Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010.

MIT SPARK Lab. VNAV 2023 Lab 3: Trajectory Tracking for UAVs. https://vnav.mit.edu/labs_2023/lab3/exercises.html, 2023a.

MIT SPARK Lab. VNAV-labs, release VNAV2023. https://github.com/MIT-SPARK/VNAV-labs/releases/tag/VNAV2023, 2023b.